

APÊNDICE

APÊNDICE A - CÓDIGO FONTE DO DIAGRAMA DE CLASSE PRODUZIDO A PARTIR DO PLANTUML

```
@startuml
skinparam classAttributeIconSize 0
```

```
enum StatusTransmissao {
    AGENDADA
    AO_VIVO
    ENCERRADA
}
```

```
enum TipoPermissao {
    ADMIN
    ORGANIZADOR
    PARTICIPANTE
}
```

```
enum StatusTreinamento {
    PENDENTE
    EM_ANDAMENTO
    CONCLUIDO
}
```

```
class Usuario {
    id: Long
    nome: String
    email: String
    senhaHash: String
    ativo: Boolean
    setorId: Long
    cargoId: Long
    createdAt: Date
    updatedAt: Date

    +autenticar()
    +atualizarPerfil()
    +desativar()
}
```

```
class Administrador {
    nivelAcesso: Integer

    +criarUsuario()
    +gerenciarPermissoes()
    +gerarRelatorio()
}
```

```
class Organizador {
```

```

    especialidade: String

    +criarTransmissao()
    +encerrarTransmissao()
    +gerenciarParticipantes()
}

class Participante {
    matricula: String

    +participarTransmissao()
    +enviarMensagem()
}

Administrador --|> Usuario
Organizador --|> Usuario
Participante --|> Usuario

interface AuthProvider {
    +autenticar(email, senha): Token
}

class GoogleAuthProvider {
    +autenticarOAuth()
    +validarToken()
}

class LDAPAuthProvider {
    +conectarServidor()
    +validarCredenciais()
}

class AuthService {
    +login(email, senha)
    +logout()
    +refreshToken()
}

class Token {
    accessToken: String
    refreshToken: String
    expiracao: Date
    tipo: String
    provedor: String
    ativo: Boolean

    +validar()
    +expirar()
}

AuthProvider <|.. GoogleAuthProvider
AuthProvider <|.. LDAPAuthProvider
AuthService --> AuthProvider

```

AuthService --> Token
 Usuario --> AuthService

```
class Setor {
  id: Long
  nome: String
```

```
  +criar()
  +editar()
  +remover()
}
```

```
class Cargo {
  id: Long
  nome: String
  +criar()
  +editar()
}
```

```
class Permissao {
  id: Long
  tipo: TipoPermissao
```

```
  +atribuir()
  +revogar()
}
```

Usuario --> Setor
 Usuario --> Cargo
 Cargo --> Permissao

```
class Transmissao {
  id: Long
  titulo: String
  dataInicio: Date
  status: StatusTransmissao
  +iniciar()
  +encerrar()
  +cancelar()
  +validarPermissao()
}
```

Organizador --> Transmissao
 Participante --> Transmissao

```
class Mensagem {
  id: Long
  conteudo: String
  dataHora: Date
```

```
  +enviar()
  +editar()
  +excluir()
}
```

Mensagem --> Usuario

Mensagem --> Transmissao

```
class Participacao {
    id: Long
    dataEntrada: Date
    dataSaida: Date
    duracao: Double

    +registrarEntrada()
    +registrarSaida()
    +calcularDuracao()
}
```

Participacao --> Usuario

Participacao --> Transmissao

```
class Auditoria {
    id: Long
    acao: String
    dataHora: Date

    +registrarAcao()
    +gerarLog()
}
```

Auditoria --> Usuario

Auditoria --> Transmissao

```
class Treinamento {
    id: Long
    titulo: String
    descricao: String
    cargaHoraria: Integer

    +criar()
    +atualizar()
    +vincularTransmissao()
}
```

```
class RegistroTreinamento {
    id: Long
    usuarioId: Long
    treinamentoId: Long
    status: StatusTreinamento
    dataInicio: Date
    dataConclusao: Date
    progresso: Double

    +iniciar()
    +concluir()
    +atualizarProgresso()
}
```

```

class TrilhaAprendizagem {
    id: Long
    nome: String
    descricao: String
    cargaTotal: Integer

    +adicionarTreinamento()
    +removerTreinamento()
}

Usuario --> RegistroTreinamento
Treinamento --> RegistroTreinamento
TrilhaAprendizagem --> Treinamento
Transmissao --> Treinamento : "base de"

class SistemaExterno {
    id: Long
    nome: String

    +conectar()
    +sincronizar()
}

class SistemaLMS {
    id: Long
    nome: String

    +enviarDadosTreinamento()
    +receberCursos()
}

class SistemaRH {
    id: Long
    nome: String
    url: String
    apiKey: String
    tipoIntegracao: String
    ativo: Boolean

    +sincronizarUsuarios()
    +sincronizarCargos()
    +sincronizarSetores()
    +enviarDadosTreinamento()
}

class APIService {
    +enviarDados()
    +receberDados()
    +autenticarAPI()
}

APIService --> SistemaExterno
APIService --> SistemaLMS
APIService --> SistemaRH

```

```
SistemaRH --> Usuario : "sincroniza dados"  
SistemaRH --> Cargo : "sincroniza cargos"  
SistemaRH --> Setor : "sincroniza setores"  
SistemaLMS --> Treinamento : "sincroniza cursos"
```

@endum1

APÊNDICE B - CÓDIGO FONTE DOS DIAGRAMAS DE SEQUÊNCIA PRODUZIDO A PARTIR DO PLANTUML

```

@startuml
actor Usuario
participant "Frontend" as FE
participant "AuthService" as Auth
participant "Google OAuth" as Google
participant "LDAP Server" as LDAP
database "Banco de Dados" as DB

Usuario -> FE : inserir credenciais / escolher login
FE -> Auth : solicitar autenticação

alt Login com Google
    Auth -> Google : validar token OAuth
    Google --> Auth : token válido
else Login com LDAP
    Auth -> LDAP : validar credenciais
    LDAP --> Auth : autenticação válida
end

Auth -> DB : verificar/criar usuário
DB --> Auth : usuário válido

Auth --> FE : retorno sucesso + token
FE --> Usuario : acesso liberado

@enduml

@startuml
actor Organizador
participant "Frontend" as FE
participant "TransmissaoService" as TS
database "Banco de Dados" as DB

Organizador -> FE : preencher dados da transmissão
FE -> TS : criarTransmissao(dados)

TS -> DB : salvar transmissão
DB --> TS : confirmação

TS --> FE : transmissão criada
FE --> Organizador : exibir sucesso

@enduml

```

```
@startuml
actor Participante
participant "Frontend" as FE
participant "TransmissaoService" as TS
participant "ChatService" as Chat
participant "ParticipacaoService" as PS
database "Banco de Dados" as DB

Participante -> FE : acessar transmissão
FE -> TS : solicitar entrada

TS -> DB : verificar transmissão
DB --> TS : dados válidos

TS -> PS : registrar entrada
PS -> DB : salvar participação
DB --> PS : confirmação

TS --> FE : acesso liberado
FE --> Participante : transmissão exibida

loop Chat em tempo real
    Participante -> FE : enviar mensagem
    FE -> Chat : enviarMensagem()
    Chat -> DB : salvar mensagem
    Chat --> FE : atualizar chat
end

Participante -> FE : sair da transmissão
FE -> PS : registrar saída
PS -> DB : atualizar duração

@enduml
```


APÊNDICE C - CÓDIGO FONTE DO DIAGRAMA DE ESTADOS PRODUZIDO A PARTIR DO PLANTUML

```

@startuml

[*] --> NaoAutenticado

NaoAutenticado --> Autenticando : login()

state Autenticando {

    [*] --> EscolhendoProvedor

    EscolhendoProvedor --> GoogleAuth : OAuth Google

    EscolhendoProvedor --> LDAPAuth : LDAP

    GoogleAuth --> Autenticado : sucesso

    LDAPAuth --> Autenticado : sucesso

    GoogleAuth --> FalhaAuth : erro

    LDAPAuth --> FalhaAuth : erro

}

FalhaAuth --> NaoAutenticado

Autenticado --> SistemaAtivo

state SistemaAtivo {

    state Transmissao {

        [*] --> Agendada

        Agendada --> AoVivo : iniciar()

        Agendada --> Cancelada : cancelar()

        state AoVivo {

        }

        note right of AoVivo

            Streaming ativo

            Chat habilitado

            Apenas participantes enviam mensagens

        end note

    }

}

```

```

    AoVivo --> Encerrada : encerrar()

    Encerrada --> [*]

    Cancelada --> [*]
}

state Participacao {

    [*] --> Fora

    Fora --> Assistindo : entrar()

    Assistindo --> Chat : enviarMensagem()

    Chat --> Assistindo

    Assistindo --> Saiu : sair()

    Saiu --> [*]
}

state Treinamento {

    [*] --> Pendente

    Pendente --> EmAndamento : iniciar()

    EmAndamento --> Concluido : finalizar()

    Concluido --> [*]
}

state Integracoes {

    [*] --> Idle

    Idle --> SyncRH : sincronizar()

    SyncRH --> Idle

    Idle --> SyncLMS : enviarDados()

    SyncLMS --> Idle
}
}

```

```
SistemaAtivo --> [*]
```

```
@endum1
```

APÊNDICE D - CÓDIGO FONTE DO DIAGRAMA DE ARQUITETURA PRODUZIDO A PARTIR DO PLANTUML

```
@startuml

skinparam componentStyle rectangle

package "Camada de Apresentação" {

    [Web App (Admin)] as Web

    [Desktop App (Transmissão)] as Desktop

}

package "Interface / API" {

    [Controllers REST] as API

    [Auth Controller]

    [Transmissao Controller]

    [Usuario Controller]

}

package "Application Layer" {

    [Auth Service]

    [Usuario Service]

    [Transmissao Service]

    [Treinamento Service]

    [Relatorio Service]

}

package "Domain Layer" {

    [Usuario]

    [Transmissao]

    [Participacao]

    [Treinamento]

    [Auditoria]

    [Permissao]
```

```

}

package "Infrastructure Layer" {

    [Banco de Dados]

    [API Externa]

    [Auth Providers]

}

package "Sistemas Externos" {

    [Google OAuth]

    [LDAP / Active Directory]

    [Sistema RH]

    [Sistema LMS]

}

' Front -> API

Web --> API

Desktop --> API

' API -> Application

API --> "Auth Service"

API --> "Usuario Service"

API --> "Transmissao Service"

' Application -> Domain

"Auth Service" --> Usuario

"Usuario Service" --> Usuario

"Transmissao Service" --> Transmissao

"Treinamento Service" --> Treinamento

"Relatorio Service" --> Auditoria

' Domain -> Infra

Usuario --> "Banco de Dados"

Transmissao --> "Banco de Dados"

```

Participacao --> "Banco de Dados"

' Auth Providers

"Auth Service" --> "Auth Providers"

' Integrações externas

"Auth Providers" --> "Google OAuth"

"Auth Providers" --> "LDAP / Active Directory"

"API Externa" --> "Sistema RH"

"API Externa" --> "Sistema LMS"

"Treinamento Service" --> "API Externa"

"Usuario Service" --> "API Externa"

@endum1

APÊNDICE E - CÓDIGO FONTE DO DIAGRAMA DE COMPONENTES PRODUZIDO A PARTIR DO PLANTUML

```
@startuml
skinparam componentStyle rectangle

package "Frontend" {
    [Web App]
    [Admin Dashboard]
}

package "Backend Services" {
    [AuthService]
    [UsuarioService]
    [TransmissaoService]
    [ChatService]
    [ParticipacaoService]
    [AuditoriaService]
    [TreinamentoService]
    [IntegracaoService]
}

package "Database" {
    [Banco de Dados]
}

package "Sistemas Externos" {
    [Google OAuth]
    [LDAP / Active Directory]
    [Sistema RH]
    [Sistema LMS]
}

[Web App] --> [AuthService]
```

[Web App] --> [TransmissaoService]
[Web App] --> [ChatService]
[Web App] --> [ParticipacaoService]
[Admin Dashboard] --> [UsuarioService]
[Admin Dashboard] --> [AuditoriaService]
[Admin Dashboard] --> [TreinamentoService]
[AuthService] --> [UsuarioService]
[TransmissaoService] --> [ParticipacaoService]
[TransmissaoService] --> [AuditoriaService]
[ChatService] --> [TransmissaoService]
[ParticipacaoService] --> [UsuarioService]
[ParticipacaoService] --> [TransmissaoService]
[TreinamentoService] --> [ParticipacaoService]
[IntegracaoService] --> [TreinamentoService]
[IntegracaoService] --> [UsuarioService]
[AuthService] --> [Banco de Dados]
[UsuarioService] --> [Banco de Dados]
[TransmissaoService] --> [Banco de Dados]
[ChatService] --> [Banco de Dados]
[ParticipacaoService] --> [Banco de Dados]
[AuditoriaService] --> [Banco de Dados]
[TreinamentoService] --> [Banco de Dados]
[AuthService] --> [Google OAuth]
[AuthService] --> [LDAP / Active Directory]
[IntegracaoService] --> [Sistema RH]
[IntegracaoService] --> [Sistema LMS]

@endum1

APÊNDICE F - CÓDIGO FONTE DO DIAGRAMA DE CASO DE USO PRODUZIDO A PARTIR DO PLANTUML

```

@startuml
left to right direction
skinparam packageStyle rectangle
actor Usuario
actor Administrador
actor Organizador
actor Participante
rectangle "Google OAuth\n(Sistema Externo)" as Google
rectangle "Servidor LDAP\n(Sistema Corporativo)" as LDAP
rectangle "Sistema Externo" as Ext
rectangle "Sistema LMS" as LMS
rectangle SICIT {
    ' AUTENTICAÇÃO
    (Autenticar Usuário) as UC0
    (Login via Google) as UC1
    (Login via LDAP) as UC2
    ' GESTÃO DE USUÁRIOS
    (Cadastrar Usuário) as UC3
    (Gerenciar Estrutura Organizacional) as UC4
    (Definir Permissões) as UC5
    ' TRANSMISSÕES
    (Criar Transmissão) as UC6
    (Participar de Transmissão) as UC7
    (Encerrar Transmissão) as UC13
    ' COMUNICAÇÃO
    (Enviar Mensagem no Chat) as UC8

```

```

' MONITORAMENTO

(Registrar Participação) as UC9

(Gerar Relatórios) as UC14

(Auditar Comunicações) as UC10

' INTEGRAÇÕES

(Integrar com Sistemas Externos) as UC11

(Integrar com LMS) as UC12

}

Usuario --> UC0

UC0 --> UC1 : <<extend>>

UC0 --> UC2 : <<extend>>

Google --> UC1

LDAP --> UC2

Administrador --> UC3

Administrador --> UC4

Administrador --> UC5

UC3 --> UC0 : <<include>>

Organizador --> UC6

Organizador --> UC13

Participante --> UC7

UC7 --> UC9 : <<include>>

UC6 --> UC0 : <<include>>

Organizador --> UC8

Participante --> UC8

UC8 --> UC7 : <<extend>>

Administrador --> UC10

Administrador --> UC14

UC10 --> UC9 : <<include>>

```

UC14 --> UC10 : <<include>>

Ext --> UC11

LMS --> UC12

UC12 --> UC9 : <<include>>

@endum1

APÊNDICE G - CÓDIGO FONTE DO DIAGRAMA BPMN PRODUZIDO A PARTIR DO PLANTUML

```
@startuml BPMN_1_Transmissao_Corporativa
```

```
skinparam backgroundColor #FFFFFF
```

```
skinparam shadowing false
```

```
skinparam defaultFontName Arial
```

```
skinparam linetype ortho
```

```
skinparam ArrowColor #111111
```

```
skinparam activity {
```

```
    BackgroundColor #F7F26B
```

```
    BorderColor #111111
```

```
    FontColor #111111
```

```
    DiamondBackgroundColor #FF9D35
```

```
    DiamondBorderColor #111111
```

```
    StartColor #C7F464
```

```
    EndColor #F06B63
```

```
}
```

```
skinparam partition {
```

```
    BackgroundColor #84D7EA
```

```
    BorderColor #111111
```

```
    FontColor #111111
```

```
}
```

```
skinparam note {
```

```
    BackgroundColor #FFFBD7
```

```
    BorderColor #111111
```

```
    FontColor #111111
```

```
}
```

```
legend right
```

Legenda BPMN visual

Bolinha verde = inicio

Bolinha vermelha = fim

Retangulo amarelo = tarefa

Diamante laranja = decisao

Nota = documento/registro

endlegend

|Usuario / Administrador|

start

:Login no SICIT;

|auth-service|

:Validar credenciais\nou Google OAuth;

if (Sessao valida?) then (Sim)

:Emitir token JWT;

else (Nao)

:Negar acesso;

|Usuario / Administrador|

:Exibir aviso de login;

stop

endif

|desktop-app / web|

:Carregar permissoes\ne dados do usuario;

|Usuario / Administrador|

:Criar live corporativa;

```
|stream-service|  
:Registrar sessao\nde transmissao;  
  
note right  
    Documento:  
    Live [Agendada]  
  
end note
```

```
|communication-service|  
:Criar canal de chat\nda live;  
  
note right  
    Documento:  
    Canal de Chat  
    [Criado]  
  
end note
```

```
|Usuario / Administrador|  
:Iniciar transmissao\nno desktop;
```

```
|desktop-app / web|  
:Capturar tela e audio;  
:Enviar stream\nvia WebSocket;
```

```
|stream-service|  
:Atualizar status\npara AO VIVO;
```

```
|Participante|  
:Entrar na live;
```

```
|stream-service|
:Registrar entrada\ndo participante;
```

```
note right
```

```
    Documento:
```

```
    Presenca [Entrada]
```

```
end note
```

```
|communication-service|
:Conectar participante\ nao chat;
```

```
|Participante|
:Enviar mensagem;
```

```
|communication-service|
:Validar conteudo\nda mensagem;
if (Mensagem inadequada?) then (Sim)
    :Bloquear mensagem;
    |desktop-app / web|
    :Exibir alerta\nSweetAlert2;
else (Nao)
    :Salvar mensagem;
    :Distribuir mensagem\nem tempo real;
endif
```

```
|Usuario / Administrador|
:Encerrar transmissao;
```

```

|stream-service|

:Atualizar status\npara ENCERRADA;

:Calcular tempo\nde permanencia;

:Salvar URL da gravacao\nquando existir;

note right

    Documento:

    Live [Encerrada]

    Presenca [Calculada]

end note

|audit-service|

:Registrar eventos\nauditaveis;

note right

    Auditoria:

    login, live, chat,

    presenca e encerramento

end note

|Usuario / Administrador|

:Consultar auditoria,\npresencas e gravacao;

stop

@enduml

title BPMN 2 - Trilha de Treinamento - SICIT

skinparam backgroundColor #FFFFFF

skinparam shadowing false

skinparam defaultFontName Arial

skinparam linetype ortho

skinparam ArrowColor #111111

skinparam activity {

    BackgroundColor #F7F26B

```



```

    BorderColor #111111

    FontColor #111111

    DiamondBackgroundColor #FF9D35

    DiamondBorderColor #111111

    StartColor #C7F464

    EndColor #F06B63
}

skinparam partition {

    BackgroundColor #84D7EA

    BorderColor #111111

    FontColor #111111
}

skinparam note {

    BackgroundColor #FFFBD7

    BorderColor #111111

    FontColor #111111
}

legend right

    <b>Legenda BPMN visual</b>

    Bolinha verde = inicio

    Bolinha vermelha = fim

    Retangulo amarelo = tarefa

    Diamante laranja = decisao

    Nota = documento/registro

endlegend

|Administrador|

start

:Login no SICIT;

```

```

:Criar trilha\nde treinamento;

:Definir setores,\netapas e percentual minimo;

|training-service|

:Registrar trilha\npublicada;

note right

    Documento:

    Trilha [Publicada]

end note

if (Moodle configurado?) then (Sim)

    |Moodle|

    :Criar curso\nno Moodle;

    :Sincronizar usuarios\ne matriculas;

    note right

        Documento:

        Curso Moodle

        [Sincronizado]

    end note

else (Nao)

    |training-service|

    :Manter trilha somente\nno SICIT;

endif

|communication-service|

:Notificar usuarios\ndos setores;

|Usuario / Colaborador|

:Acessar trilha;

:Realizar atividades\nou assistir lives;

|stream-service|

:Fornecer dados\nde presenca;

```

```

|training-service|

:Calcular progresso\ndo usuario;

if (Progresso suficiente?) then (Sim)

    :Marcar usuario\ncomo aprovado;

else (Nao)

    :Manter progresso\nparcial;

    |Usuario / Colaborador|

    :Continuar atividades;

    stop

endif

|training-service|

:Emitir certificado;

note right

    Documento:

    Certificado

    [Disponivel]

end note

|communication-service|

:Enviar notificacao\nde certificado;

|audit-service|

:Registrar eventos\nauditaveis;

note right

    Auditoria:

    criacao da trilha,

    sincronizacao Moodle,

    progresso e certificado

end note

|Usuario / Colaborador|

```

:Baixar certificado;

|Administrador|

:Consultar resultados\ne dashboard;

stop

@endum1

APÊNDICE H - CÓDIGO FONTE DO DIAGRAMA DE MAPEAMENTO OBJETO RELACIONAL PRODUZIDO A PARTIR DO PLANTUML

```

@startuml

skinparam backgroundColor #FFFFFF

skinparam classAttributeIconSize 0

skinparam shadowing false

skinparam linetype ortho

skinparam class {
    BackgroundColor #F7FAFC
    BorderColor #32373D
    ArrowColor #1B5D8A
    FontColor #32373D
}

skinparam package {
    BackgroundColor #EEF3F8
    BorderColor #6F86A3
    FontColor #32373D
}

legend right
    <b>Legenda</b>

    PK = Chave primaria

    FK = Chave estrangeira

    UQ = Restricao unique

    EXT = Referencia logica entre servicos
endlegend

package "auth-service\nPrisma / PostgreSQL" as AuthService {
    class Tenant <<table: tenants>> {
        +id: String <<PK, uuid>>
    }
}

```

```

+companyName: String <<column: company_name>>

+domain: String <<UQ>>

+status: String = "ACTIVE"

+createdAt: DateTime <<column: created_at>>

--

admins: TenantAdmin[]

ldapConfigs: TenantLdapConfig[]

googleUsers: GoogleUser[]

hrIntegrationConfigs: TenantHrIntegrationConfig[]
}

class TenantAdmin <<table: tenant_admins>> {

+id: String <<PK, uuid>>

+tenantId: String <<FK, column: tenant_id>>

+name: String

+email: String

+passwordHash: String <<column: password_hash>>

+role: String = "TENANT_ADMIN"

+createdAt: DateTime <<column: created_at>>

--

unique(tenantId, email)

}

class TenantLdapConfig <<table: tenant_ldap_configs>> {

+id: String <<PK, uuid>>

+tenantId: String <<FK, column: tenant_id>>

+ldapUrl: String <<column: ldap_url>>

+baseDn: String <<column: base_dn>>

+bindDn: String <<column: bind_dn>>

```

```

+bindPassword: String <<column: bind_password>>

+userAttribute: String = "uid" <<column: user_attribute>>

+isActive: Boolean = true <<column: is_active>>

+createdAt: DateTime <<column: created_at>>

}

```

```

class GoogleUser <<table: google_users>> {

  +id: String <<PK, uuid>>

  +googleId: String <<UQ, column: google_id>>

  +name: String?

  +email: String <<UQ>>

  +picture: String?

  +role: String = "GOOGLE_USER"

  +createdAt: DateTime <<column: created_at>>

  +updatedAt: DateTime <<column: updated_at>>

  +tenantId: String? <<FK, column: tenant_id>>

}

```

```

class TenantHrIntegrationConfig <<table: tenant_hr_integration_configs>> {

  +id: String <<PK, uuid>>

  +tenantId: String <<FK, column: tenant_id>>

  +provider: String = "CUSTOM_API"

  +apiUrl: String <<column: api_url>>

  +apiToken: String <<column: api_token>>

  +employeesEndpoint: String <<column: employees_endpoint>>

  +sectorsEndpoint: String <<column: sectors_endpoint>>

  +rolesEndpoint: String <<column: roles_endpoint>>

  +syncEmployees: Boolean = true <<column: sync_employees>>

```

```

+syncRoles: Boolean = true <<column: sync_roles>>

+syncSectors: Boolean = true <<column: sync_sectors>>

+frequency: String = "Diaria"

+isActive: Boolean = true <<column: is_active>>

+createdAt: DateTime <<column: created_at>>

+updatedAt: DateTime <<column: updated_at>>

}

}

package "user-service\nPrisma / PostgreSQL" as UserService {

  class Role <<table: roles>> {

    +id: String <<PK, uuid>>

    +tenantId: String <<EXT, column: tenant_id>>

    +name: String

    +description: String?

    +createdAt: DateTime <<column: created_at>>

    --

    users: User[]

    rolePermissions: RolePermission[]

    unique(tenantId, name)

  }

  class Sector <<table: sectors>> {

    +id: String <<PK, uuid>>

    +tenantId: String <<EXT, column: tenant_id>>

    +name: String

    +description: String?

    +createdAt: DateTime <<column: created_at>>

    --

```



```

    users: User[]

    unique(tenantId, name)
}

```

```

class User <<table: users>> {

    +id: String <<PK, uuid>>

    +tenantId: String <<EXT, column: tenant_id>>

    +name: String

    +email: String

    +roleId: String? <<FK, column: role_id>>

    +sectorId: String? <<FK, column: sector_id>>

    +externalId: String? <<column: external_id>>

    +source: String = "LOCAL"

    +status: String = "ACTIVE"

    +lastSyncAt: DateTime? <<column: last_sync_at>>

    +createdAt: DateTime <<column: created_at>>

    --

    unique(tenantId, email)
}

```

```

class Permission <<table: permissions>> {

    +id: String <<PK, uuid>>

    +code: String <<UQ>>

    +name: String

    +description: String?

    +createdAt: DateTime <<column: created_at>>

    --

    rolePermissions: RolePermission[]
}

```

```

}

class RolePermission <<table: role_permissions>> {
  +id: String <<PK, uuid>>
  +roleId: String <<FK, column: role_id>>
  +permissionId: String <<FK, column: permission_id>>
  +createdAt: DateTime <<column: created_at>>
  --
  unique(roleId, permissionId)
}
}

' Auth service ORM relations

Tenant "1" o-- "0..*" TenantAdmin : admins
Tenant "1" o-- "0..*" TenantLdapConfig : ldapConfigs
Tenant "1" o-- "0..*" GoogleUser : googleUsers
Tenant "1" o-- "0..*" TenantHrIntegrationConfig : hrIntegrationConfigs
TenantAdmin "*" --> "1" Tenant : tenantId
TenantLdapConfig "*" --> "1" Tenant : tenantId
GoogleUser "0..*" --> "0..1" Tenant : tenantId
TenantHrIntegrationConfig "*" --> "1" Tenant : tenantId

' User service ORM relations

Role "1" o-- "0..*" User : users
Sector "1" o-- "0..*" User : users
Role "1" o-- "0..*" RolePermission : rolePermissions
Permission "1" o-- "0..*" RolePermission : rolePermissions
User "0..*" --> "0..1" Role : roleId / SetNull
User "0..*" --> "0..1" Sector : sectorId / SetNull
RolePermission "*" --> "1" Role : roleId / Cascade

```

RolePermission "*" --> "1" Permission : permissionId / Cascade

' Logical cross-service references by tenantId

Tenant ..> Role : tenantId\nreferencia logica

Tenant ..> Sector : tenantId\nreferencia logica

Tenant ..> User : tenantId\nreferencia logica

note bottom of UserService

No user-service, tenantId nao possui @relation no Prisma.

Ele representa uma referencia logica ao Tenant do auth-service.

end note

@enduml

APÊNDICE I - CÓDIGO FONTE DOS SCRIPT DO BANCO DE DADOS DE AUTH-SERVICE

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model Tenant {
  id          String          @id @default(uuid())
  companyName String          @map("company_name")
  domain      String          @unique
  status      String          @default("ACTIVE")
  createdAt   DateTime        @default(now()) @map("created_at")

  admins      TenantAdmin[]
  ldapConfigs TenantLdapConfig[]
  googleUsers GoogleUser[]
  hrIntegrationConfigs TenantHrIntegrationConfig[]

  @@map("tenants")
}

model TenantAdmin {
  id          String  @id @default(uuid())
  tenantId    String  @map("tenant_id")
  name        String
  email       String

```

```

passwordHash String    @map("password_hash")

role           String    @default("TENANT_ADMIN")

createdAt      DateTime @default(now()) @map("created_at")

tenant Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)

@@unique([tenantId, email])

@@map("tenant_admins")
}

model TenantLdapConfig {
  id          String    @id @default(uuid())
  tenantId     String    @map("tenant_id")
  ldapUrl      String    @map("ldap_url")
  baseDn       String    @map("base_dn")
  bindDn       String    @map("bind_dn")
  bindPassword String    @map("bind_password")
  userAttribute String    @default("uid") @map("user_attribute")
  isActive     Boolean    @default(true) @map("is_active")
  createdAt    DateTime @default(now()) @map("created_at")
  tenant Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)

  @@map("tenant_ldap_configs")
}

model GoogleUser {
  id          String    @id @default(uuid())
  googleId     String    @unique @map("google_id")
  name         String?
  email        String    @unique
  picture      String?
  role         String    @default("GOOGLE_USER")
  createdAt    DateTime @default(now()) @map("created_at")

```

```

    updatedAt    DateTime @updatedAt @map("updated_at")

    tenantId String? @map("tenant_id")

    tenant    Tenant? @relation(fields: [tenantId], references: [id])
    @@map("google_users")
}

model TenantHrIntegrationConfig {
  id          String  @id @default(uuid())
  tenantId    String  @map("tenant_id")
  provider    String  @default("CUSTOM_API")
  apiUrl      String  @map("api_url")
  apiToken    String  @map("api_token")
  employeesEndpoint String @map("employees_endpoint")
  sectorsEndpoint String @map("sectors_endpoint")
  rolesEndpoint String @map("roles_endpoint")
  syncEmployees Boolean @default(true) @map("sync_employees")
  syncRoles   Boolean @default(true) @map("sync_roles")
  syncSectors Boolean @default(true) @map("sync_sectors")
  frequency   String  @default("Diaria")
  isActive    Boolean @default(true) @map("is_active")
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")
  tenant      Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  @@map("tenant_hr_integration_configs")
}

```

APÊNDICE J - CÓDIGO FONTE DOS SCRIPT DO BANCO DE DADOS DE USER-SERVICE

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model Role {
  id          String   @id @default(uuid())
  tenantId    String   @map("tenant_id")
  name        String
  description  String?
  createdAt   DateTime @default(now()) @map("created_at")
  users       User[]
  rolePermissions RolePermission[]
  @@unique([tenantId, name])
  @@map("roles")
}

model Sector {
  id          String   @id @default(uuid())
  tenantId    String   @map("tenant_id")
  name        String
  description  String?
  createdAt   DateTime @default(now()) @map("created_at")

```

```

    users User[]

    @@unique([tenantId, name])

    @@map("sectors")
}

model User {
    id          String    @id @default(uuid())
    tenantId    String    @map("tenant_id")
    name        String
    email       String
    roleId      String?   @map("role_id")
    sectorId    String?   @map("sector_id")
    externalId  String?   @map("external_id")
    source      String    @default("LOCAL")
    status      String    @default("ACTIVE")
    lastSyncAt  DateTime? @map("last_sync_at")
    createdAt   DateTime  @default(now()) @map("created_at")
    role        Role?     @relation(fields: [roleId], references: [id], onDelete: SetNull)
    sector      Sector?   @relation(fields: [sectorId], references: [id], onDelete:
SetNull)

    @@unique([tenantId, email])

    @@map("users")
}

model Permission {
    id          String    @id @default(uuid())
    code        String    @unique
    name        String
    description  String?

```



```

    createdAt    DateTime @default(now()) @map("created_at")

    rolePermissions RolePermission[]

    @@map("permissions")
}

model RolePermission {
    id            String    @id @default(uuid())
    roleId        String    @map("role_id")
    permissionId String    @map("permission_id")
    createdAt     DateTime @default(now()) @map("created_at")
    role          Role      @relation(fields: [roleId], references: [id], onDelete: Cascade)
    permission Permission @relation(fields: [permissionId], references: [id], onDelete: Cascade)

    @@unique([roleId, permissionId])

    @@map("role_permissions")
}

```

APÊNDICE K - CÓDIGO FONTE DO DOCKER COMPOSE DE INFRAESTRUTURA DO PROJETO

services:

postgres:

image: postgres:16

container_name: sicit-postgres

restart: unless-stopped

environment:

POSTGRES_USER: sicit_user

POSTGRES_PASSWORD: sicit_pass

POSTGRES_DB: sicit_db

ports:

- "5433:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

networks:

- sicit-network

mongo:

image: mongo:7

container_name: sicit-mongo

restart: unless-stopped

environment:

MONGO_INITDB_ROOT_USERNAME: admin

MONGO_INITDB_ROOT_PASSWORD: admin123

MONGO_INITDB_DATABASE: sicit_logs

ports:

- "27017:27017"

volumes:

- mongo_data:/data/db

networks:

- sicit-network

mongo-express:

image: mongo-express:1.0.2

container_name: sicit-mongo-express

restart: unless-stopped

depends_on:

- mongo

environment:

ME_CONFIG_MONGODB_ADMINUSERNAME: admin

ME_CONFIG_MONGODB_ADMINPASSWORD: admin123

ME_CONFIG_MONGODB_SERVER: mongo

ME_CONFIG_BASICAUTH_USERNAME: admin

ME_CONFIG_BASICAUTH_PASSWORD: admin123

ports:

- "8081:8081"

networks:

- sicit-network

rabbitmq:

image: rabbitmq:3-management

container_name: sicit-rabbitmq

restart: unless-stopped

environment:

RABBITMQ_DEFAULT_USER: admin

RABBITMQ_DEFAULT_PASS: admin123

ports:

- "5672:5672"
- "15672:15672"

volumes:

- rabbitmq_data:/var/lib/rabbitmq

networks:

- sicit-network

lldap:

image: lldap/llldap:stable

container_name: sicit-llldap

restart: unless-stopped

ports:

- "3890:3890"
- "17170:17170"

environment:

LLDAP_JWT_SECRET: "sicit_jwt_secret_123456789"

LLDAP_KEY_SEED: "sicit_key_seed_123456789"

LLDAP_LDAP_BASE_DN: "dc=sicit,dc=local"

LLDAP_LDAP_USER_DN: "admin"

LLDAP_LDAP_USER_PASS: "admin12345"

volumes:

- lldap_data:/data

networks:

- sicit-network

moodle-db:

```
image: mysql:5.7

container_name: sicit-moodle-db

restart: unless-stopped

environment:

    MYSQL_ROOT_PASSWORD: root123

    MYSQL_DATABASE: moodle

    MYSQL_USER: moodle

    MYSQL_PASSWORD: moodle123

volumes:

    - moodle_db:/var/lib/mysql

networks:

    - sicit-network
```

moodle:

```
image: aesr/moodle

container_name: sicit-moodle

restart: unless-stopped

depends_on:

    - moodle-db

ports:

    - "8090:80"

environment:

    MOODLE_DB_HOST: moodle-db

    MOODLE_DB_PORT: 3306

    MOODLE_DB_NAME: moodle

    MOODLE_DB_USER: moodle

    MOODLE_DB_PASSWORD: moodle123

    MOODLE_WWW_ROOT: http://localhost:8090
```

```
MOODLE_DATA_ROOT: /var/www/moodledata
```

```
volumes:
```

- moodle_data:/var/www/html
- moodle_moodledata:/var/www/moodledata

```
networks:
```

- sicit-network

```
volumes:
```

```
postgres_data:
```

```
mongo_data:
```

```
rabbitmq_data:
```

```
lldap_data:
```

```
moodle_db:
```

```
moodle_data:
```

```
moodle_moodledata:
```

```
networks:
```

```
sicit-network:
```

```
driver: bridge
```

APÊNDICE L - CÓDIGO FONTE DOS COMANDOS PARA EXECUÇÃO DOS TESTES

```
```bash
```

```
npm test
```

```
npm run test:unit
```

```
npm run test:integration
```

```
npm run test:contract
```

```
npm run test:e2e
```

```
npm run test:performance
```

```
npm run test:load
```

```
npm run test:security
```

```
npm run test:iam
```

```
npm run test:data-integrity
```

```
npm run test:coverage
```

```
```
```

APÊNDICE M - CÓDIGO FONTE DO TESTE PARA AUTORIZAÇÃO E AUTENTICAÇÃO

```
import request from "supertest";

import fs from "fs";

import path from "path";

import { createProtectedTestApp } from "../helpers/createProtectedTestApp.js";
import { createPermissionTestApp } from "../helpers/createPermissionTestApp.js";
import { createTestToken } from "../helpers/testToken.js";

const LogPath = path.resolve("tests/results/iam-access-log.txt");

function writeLog({ scenario, method, endpoint, status, body }) {
  const line = [
    `[${new Date().toISOString()}]`,
    `scenario="${scenario}"`,
    `method=${method}`,
    `endpoint=${endpoint}`,
    `status=${status}`,
    `body=${JSON.stringify(body)}`,
  ].join(" ");

  fs.appendFileSync(LogPath, `${line}\n`, "utf8");

  console.log(line);
}

describe("autenticacao e autorizacao", () => {
  beforeAll(() => {
    fs.mkdirSync(path.dirname(LogPath), { recursive: true });
  });
});
```



```

fs.writeFileSync(
  LogPath,
  "IAM access evidence log - SICIT\n",
  "utf8"
);
});

it("usuario autenticado ao acessar recurso permitido", async () => {
  const app = createProtectedTestApp({
    method: "get",
    path: "/auth/me",
    handler: (req, res) =>
      res.status(200).json({
        status: "AUTHORIZED",
        user: {
          id: req.user.id,
          tenantId: req.user.tenantId,
          email: req.user.email,
          role: req.user.role,
        },
      }),
  });

  const token = createTestToken({
    id: "0de43ea9-a988-47f9-959e-ec055738ee7a",
    tenantId: "f4363723-5344-4523-a232-2b0f1a7c9001",
    email: "admin.sicit@empresa.com",
    role: "ADMIN",
  });

```

```

const response = await request(app)

  .get("/auth/me")

  .set("Authorization", `Bearer ${token}`);

writelog({
  scenario: "usuario autenticado autorizado",
  method: "GET",
  endpoint: "/auth/me",
  status: response.status,
  body: response.body,
});

expect(response.status).toBe(200);
expect(response.body).toEqual({
  status: "AUTHORIZED",
  user: {
    id: "0de43ea9-a988-47f9-959e-ec055738ee7a",
    tenantId: "f4363723-5344-4523-a232-2b0f1a7c9001",
    email: "admin.sicit@empresa.com",
    role: "ADMIN",
  },
});

});

it("usuario nao autenticado", async () => {
  const app = createProtectedTestApp({
    method: "get",

```

```

    path: "/auth/me",

    handler: (_req, res) => res.status(200).json({ status: "AUTHORIZED" }),
  });

const response = await request(app).get("/auth/me");

writeLog({
  scenario: "usuario nao autenticado barrado",
  method: "GET",
  endpoint: "/auth/me",
  status: response.status,
  body: response.body,
});

expect(response.status).toBe(401);
expect(response.body).toHaveProperty("message");
});

it("usuario autenticado sem permissao necessaria", async () => {
  const app = createPermissionTestApp();

  const token = createTestToken({
    id: "7a5e0c8f-8d3e-46aa-9c8b-0f0e9e4d2a11",
    tenantId: "f4363723-5344-4523-a232-2b0f1a7c9001",
    email: "colaborador.sicit@empresa.com",
    role: "EMPLOYEE",
    permissions: [],
  });
});

```

```
const response = await request(app)

  .post("/users/users")

  .set("Authorization", `Bearer ${token}`)

  .send({

    name: "Usuario Sem Permissao",

    email: "sem-permissao@sicit.local",

  });

writeLog({

  scenario: "usuario autenticado sem permissao barrado",

  method: "POST",

  endpoint: "/users/users",

  status: response.status,

  body: response.body,

});

expect(response.status).toBe(403);

expect(response.body).toHaveProperty("message");

});

});
```

APÊNDICE N - CÓDIGO FONTE DO TESTE PARA CARGA DE ENVIO DE MENSAGENS

```
import request from "supertest";

import { createCommunicationTestApp } from
"../../helpers/createCommunicationTestApp.js";

import { createTestToken } from "../../helpers/testToken.js";

describe("Carga de envio de mensagens", () => {

  it("requisicoes concorrentes sem erro", async () => {

    const app = createCommunicationTestApp();

    const token = createTestToken();

    const concurrentRequests = 300;

    const responses = await Promise.all(

      Array.from({ length: concurrentRequests }, (_, index) =>

        request(app)

          .post("/communication/channels/channel-001/messages")

          .set("Authorization", `Bearer ${token}`)

          .send({

            content: `Mensagem concorrente ${index + 1}`,

          })

      )

    );

    expect(responses).toHaveLength(concurrentRequests);

    expect(responses.every((response) => response.status === 201)).toBe(true);

  });

});
```

APÊNDICE O - CÓDIGO FONTE DO TESTE DE DESEMPENHO NO ENVIO DE MENSAGENS

```
import request from "supertest";

import { createCommunicationTestApp } from
"../../helpers/createCommunicationTestApp.js";

import { createTestToken } from "../../helpers/testToken.js";

describe("Performance de mensagens", () => {

  it("tempo medio em varias requisicoes sequenciais", async () => {

    const app = createCommunicationTestApp();

    const token = createTestToken();

    const totalRequests = 200;

    const startedAt = Date.now();

    for (let index = 0; index < totalRequests; index += 1) {

      const response = await request(app)

        .post("/communication/channels/channel-001/messages")

        .set("Authorization", `Bearer ${token}`)

        .send({

          content: `Mensagem de performance ${index + 1}`,

        });

      expect(response.status).toBe(201);

    }

    const elapsedMs = Date.now() - startedAt;

    const averageMs = elapsedMs / totalRequests;
```

```
        expect(averageMs).toBeLessThan(250);  
    });  
});
```

APÊNDICE P - CÓDIGO FONTE DO TESTE DE SEGURANÇA POR PERMISSÃO

```
import request from "supertest";

import { createPermissionTestApp } from
"../../helpers/createPermissionTestApp.js";

import { createTestToken } from "../../helpers/testToken.js";

describe("Seguranca - permissoes por cargo", () => {

  it("o token possui a permissao exigida", async () => {

    const app = createPermissionTestApp();

    const token = createTestToken({

      permissions: ["CREATE_USER"],

    });

    const response = await request(app)

      .post("/users/users")

      .set("Authorization", `Bearer ${token}`)

      .send({

        name: "Usuario Autorizado",

        email: "autorizado@sicit.local",

      });

    expect(response.status).toBe(201);

    expect(response.body).toEqual(

      expect.objectContaining({

        name: "Usuario Autorizado",

        email: "autorizado@sicit.local",

      })

    );

  });

});
```



```
});

it("o token nao possui a permissao exigida", async () => {

  const app = createPermissionTestApp();

  const token = createTestToken({

    permissions: [],

  });

  const response = await request(app)

    .post("/users/users")

    .set("Authorization", `Bearer ${token}`)

    .send({

      name: "Usuario Sem Permissao",

      email: "sem-permissao@sicit.local",

    });

  expect(response.status).toBe(403);

  expect(response.body).toHaveProperty("message");

});

});
```

APÊNDICE Q - CÓDIGO FONTE DO TESTE DE PONTA A PONTA PARA LIVE

8.3.6 Exemplo de teste ponta a ponta para live

```
import request from "supertest";

import { createCommunicationTestApp } from
"../helpers/createCommunicationTestApp.js";

import { createTestToken } from "../helpers/testToken.js";

import { communicationFixtures } from "../fixtures/communication.js";

import { makeMessagePayload } from "../factories/messageFactory.js";

describe("Fluxo principal da live corporativa", () => {

  it("permite que um colaborador autenticado participe do chat da live", async ()
=> {

    const app = createCommunicationTestApp();

    const token = createTestToken(communicationFixtures.employeeUser);

    const response = await request(app)

.post(`/communication/channels/${communicationFixtures.channel.id}/messages`)

  .set("Authorization", `Bearer ${token}`)

  .send(makeMessagePayload());

    expect(response.status).toBe(201);

    expect(response.body).toMatchObject({

      channelId: communicationFixtures.channel.id,

      senderId: communicationFixtures.employeeUser.id,

      senderName: communicationFixtures.employeeUser.name,

      type: "text",

    });

  });

});
```

APÊNDICE R - IMAGENS DOS LOGS DE TESTES GERAIS

```
(node:20076) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS contract tests/contracts/communication/send-message.contract.test.js
Contrato da API de mensagens da live
  ✓ mantém o formato publico esperado ao enviar mensagem (38 ms)
  ✓ mantém contrato de erro para requisicao sem token (5 ms)
  ✓ mantém contrato de erro para corpo invalido (6 ms)

console.log
  [ ] injected env (0) from .env // tip: [ ] enable debugging { debug: true }

    at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
PASS load tests/load/communication/send-message.load.test.js
Carga - envio de mensagens
  ✓ processa varias requisicoes concorrentes sem erro (92 ms)

console.log
  [ ] injected env (0) from .env // tip: [ ] suppress logs { quiet: true }

    at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
PASS performance tests/performance/communication/send-message.performance.test.js
Performance - envio de mensagens
  ✓ mantém tempo medio aceitavel em varias requisicoes sequenciais (70 ms)

console.log
  [ ] injected env (0) from .env // tip: [ ] multiple files { path: ['.env.local', '.env'] }

    at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
PASS integration tests/integration/communication/send-message.integration.test.js
POST /communication/channels/:channelId/messages
  ✓ retorna 201 quando o token e o corpo da requisicao sao validos (21 ms)
  ✓ retorna 401 quando o token nao e enviado (3 ms)
  ✓ retorna 400 quando o corpo da requisicao e invalido (4 ms)
  ✓ usa email como nome do remetente quando name nao vem no token (3 ms)
  ✓ retorna 400 quando content nao e enviado (4 ms)
  ✓ retorna 401 quando authorization usa formato incorreto (3 ms)

console.log
  [ ] injected env (0) from .env // tip: [ ] override existing { override: true }

    at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
PASS e2e tests/e2e/corporate-live-flow.e2e.test.js
Fluxo principal da live corporativa
  ✓ permite que um colaborador autenticado participe do chat da live (22 ms)

console.log
  [ ] injected env (0) from .env // tip: [ ] encrypted .env [www.dotenvx.com]

    at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
```



```

PASS integration tests/integration/user/sectors.integration.test.js
  POST /users/sectors
    ✓ cria setor usando tenant do usuario autenticado (19 ms)

    console.log
      [ ] injected env (0) from .env // tip: [ ] suppress logs { quiet: true }

      at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
PASS integration tests/integration/training/paths.integration.test.js
  POST /training/paths
    ✓ cria trilha vinculada ao tenant autenticado (21 ms)

    console.log
      [ ] injected env (0) from .env // tip: [ ] suppress logs { quiet: true }

      at _log (services/communication-service/node_modules/dotenv/lib/main.js:131:11)
PASS integration tests/integration/analytics/dashboard.integration.test.js
  GET /analytics/dashboard/executive
    ✓ retorna indicadores consolidados para usuario autenticado (12 ms)
PASS integration tests/integration/gateway/health.integration.test.js
  GET /health gateway
    ✓ retorna status resumido do gateway em aplicacao de teste (10 ms)
    ✓ retorna informacoes dos servicos monitorados (3 ms)
    ✓ permite identificar gateway em estado degradado (2 ms)
PASS unit tests/unit/gateway/circuit-breaker-config.unit.test.js
  Gateway circuit breaker
    ✓ expoe status dos circuit breakers em formato de objeto (2 ms)
PASS unit tests/unit/user/CreateSectorUseCase.unit.test.js
  CreateSectorUseCase
    ✓ cria setor quando nome ainda nao existe no tenant (3 ms)
    ✓ bloqueia setor duplicado no mesmo tenant (4 ms)
    ✓ permite setores com mesmo nome em tenants diferentes (1 ms)
    ✓ bloqueia criacao sem tenant (3 ms)
    ✓ bloqueia criacao sem nome
    ✓ mantem descricao nula quando nao informada (1 ms)
PASS data-integrity tests/data-integrity/communication/message.data-integrity.test.js
  Integridade de dados - mensagens
    ✓ garante campos obrigatorios e padroes ao persistir mensagem (3 ms)
    ✓ nao permite persistir mensagem sem campos obrigatorios (5 ms)
PASS unit tests/unit/stream/CreateStreamSessionUseCase.unit.test.js
  CreateStreamSessionUseCase
    ✓ cria live agendada e registra host como participante (3 ms)
    ✓ cria live sem canal de chat quando nao recebe token (1 ms)
    ✓ usa visibilidade publica como padrao (1 ms)
    ✓ bloqueia criacao sem titulo (5 ms)
    ✓ bloqueia criacao sem host (1 ms)
PASS unit tests/unit/audit/CreateAuditEventUseCase.unit.test.js
  CreateAuditEventUseCase
    ✓ registra evento auditavel com metadata padrao (2 ms)
    ✓ registra metadata e resourceId quando informados (1 ms)
    ✓ bloqueia evento sem tenant (5 ms)
    ✓ bloqueia evento sem usuario (1 ms)
    ✓ bloqueia evento sem acao (1 ms)

```

PASS unit tests/unit/communication/SendMessageUseCase.unit.test.js

SendMessageUseCase

- ✓ cria uma mensagem quando o usuario participa do canal (4 ms)
- ✓ bloqueia mensagem quando o usuario nao participa do canal (4 ms)
- ✓ bloqueia mensagem sem conteudo (2 ms)
- ✓ bloqueia mensagem com palavra inadequada (1 ms)
- ✓ funciona sem callbacks de publicacao e socket (1 ms)
- ✓ persiste mensagens por canal no repositorio em memoria (2 ms)

PASS unit tests/unit/analytics/GetStreamKpisUseCase.unit.test.js

GetStreamKpisUseCase

- ✓ consolida indicadores de transmissoes e eventos (3 ms)
- ✓ retorna indicadores zerados quando nao existem lives nem eventos (1 ms)
- ✓ ignora eventos que nao fazem parte dos indicadores de live (1 ms)
- ✓ repassa token para os clients externos (1 ms)

PASS unit tests/unit/training/CreateTrainingUseCase.unit.test.js

CreateTrainingUseCase

- ✓ cria treinamento com percentual minimo padrao (3 ms)
- ✓ respeita percentual minimo informado (1 ms)
- ✓ cria treinamento sem stream vinculado (1 ms)
- ✓ gera identificador para cada treinamento criado (1 ms)

PASS unit tests/unit/auth/GetMeUseCase.unit.test.js

GetMeUseCase

- ✓ retorna os dados do usuario autenticado (2 ms)
- ✓ retorna null quando nao existe usuario autenticado (1 ms)
- ✓ mantem permissoes e dados extras do token (1 ms)

| file | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---|---------|----------|---------|---------|-------------------|
| all files | 100 | 97.95 | 100 | 100 | |
| analytics-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| GetStreamKpisUseCase.js | 100 | 100 | 100 | 100 | |
| audit-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| CreateAuditEventUseCase.js | 100 | 100 | 100 | 100 | |
| audit-service/src/domain/entities | 100 | 50 | 100 | 100 | |
| AuditEvent.js | 100 | 50 | 100 | 100 | 10 |
| auth-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| GetMeUseCase.js | 100 | 100 | 100 | 100 | |
| communication-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| SendMessageUseCase.js | 100 | 100 | 100 | 100 | |
| communication-service/src/domain/entities | 100 | 100 | 100 | 100 | |
| Message.js | 100 | 100 | 100 | 100 | |
| stream-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| CreateStreamSessionUseCase.js | 100 | 100 | 100 | 100 | |
| training-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| CreateTrainingUseCase.js | 100 | 100 | 100 | 100 | |
| user-service/src/application/use-cases | 100 | 100 | 100 | 100 | |
| CreateSectorUseCase.js | 100 | 100 | 100 | 100 | |

Test Suites: 23 passed, 23 total

Tests: 64 passed, 64 total

Snapshots: 0 total

Time: 5.152 s

Ran all test suites in 9 projects.

Test results written to: tests\results\coverage-results.json